Universidad de Los Andes

Facultad de Ingeniería

Entrega 2 Proyecto FBD

Docente

Melissa Carolina Castañeda Prieto

Actividad

Entrega 2 Proyecto FBD 2026-1

Estudiantes

Juan Felipe Londoño Guerrero – 202425071

Juan Esteban Gómez – 202514278

Isabella Casallas Abril – 202510253

Fundamentos de Bases de datos

Dann-Alpes. Entrega 2.

1. Análisis y revisión de modelo de datos

El modelo conceptual E/R y el modelo relacional presentados en la Entrega 1 obtuvieron una calificación de 5/5, lo que indica que el diseño cumplió satisfactoriamente con todos los criterios evaluados por el monitor. Por esta razón, **no** se realizaron ajustes ni modificaciones al modelo conceptual ni al modelo relacional en esta entrega.

a) Dado que la retroalimentación del monitor no señaló errores ni inconsistencias en el modelo E/R presentado, no fue necesario actualizar ni crear nuevas clases en esta entrega. El modelo conceptual permanece idéntico al entregado en la Entrega 1, el cual incluye las siguientes entidades correctamente modeladas:

- **Ciudad:** representa las ubicaciones geográficas de los hoteles.
- **Hotel:** cada establecimiento físico de la cadena Dann-Alpes.
- **Tipo_Habitacion:** catálogo global de tipos de habitación.
- **Tipo_Habitacion_Por_Hotel:** configuración específica de precios y disponibilidad por hotel.
- **Habitacion:** habitaciones físicas concretas dentro de cada hotel.
- **Servicios** (supertipo): servicios adicionales ofrecidos por la cadena.
- **Servicio_Costo_Fijo** (subtipo): servicios con precio fijo.
- **Servicio_Costo_Consumo** (subtipo): servicios con precio por unidad consumida.
- **HotelTipoHabitacion_Servicio:** servicios habilitados por tipo de habitación en cada hotel.
- **Cliente:** usuarios que realizan reservas.
- **Administrador:** usuarios con permisos de gestión.
- **Reservas:** cada reserva realizada por un cliente.
- **Reserva_Servicio:** servicios adicionales seleccionados dentro de una reserva.
- **Administrado_Por:** relación N:M entre administradores y hoteles.

b) **Modelo Relacional**

De igual manera, dado que el modelo relacional obtuvo la máxima calificación en la Entrega 1, no se realizaron modificaciones a las tablas, atributos, claves primarias, claves foráneas ni restricciones definidas originalmente. El modelo relacional implementado en esta entrega corresponde fielmente al diseño aprobado, el cual fue transformado siguiendo el algoritmo de Chen con las siguientes reglas:

- Cada entidad fuerte se transformó en una tabla independiente conservando su clave primaria.
- Las relaciones 1:N se resolvieron incorporando la clave primaria del lado "1" como clave foránea en la tabla del lado "N".
- Las relaciones N:M se resolvieron creando tablas intermedias con clave primaria compuesta.
- La herencia del supertipo Servicios se implementó con la Alternativa 1 de Chen, generando dos tablas independientes: DA_SERVICIO_COSTO_FIJO y DA_SERVICIO_COSTO_CONSUMO.

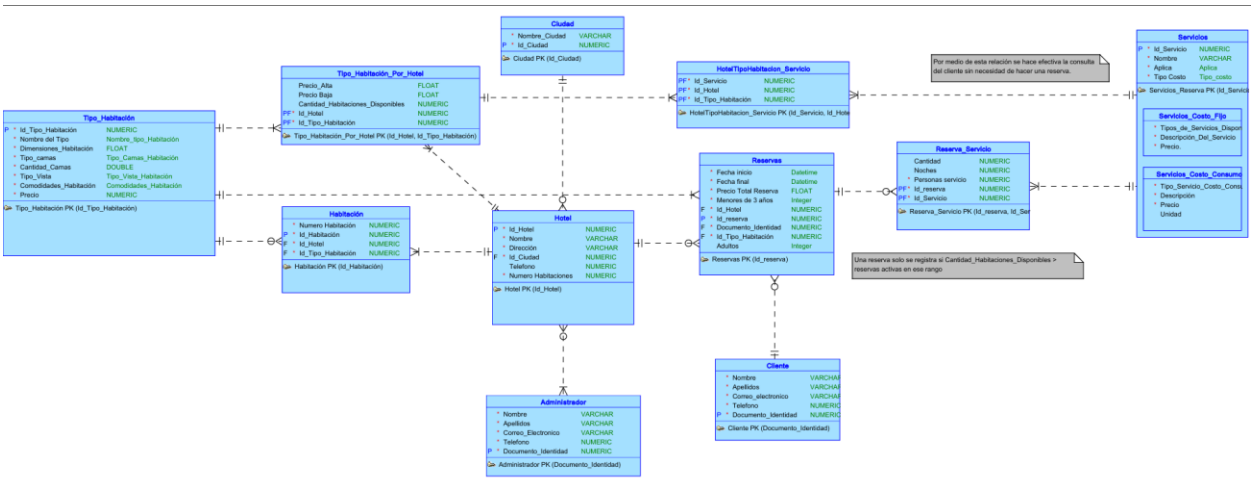
c) **Verificación de la Forma Normal de Boyce-Codd (BCNF)**
Dado que el modelo fue aprobado con la máxima calificación, se confirma que todas las relaciones del modelo ya cumplen con la **Forma Normal de Boyce-Codd (BCNF)** desde la Entrega 1. A continuación se resume el cumplimiento por tabla:

Tabla	Cumple BCNF	Justificación
DA_CIUDAD	Si	ID_Ciudad determina todos los atributos. No hay dependencias parciales ni transitivas.
DA_HOTEL	Si	ID_Hotel determina todos los atributos. FK ID_Ciudad no genera dependencia transitiva.
DA_TIPO_HABITACION	Si	ID_Tipo_Habitacion determina todos los atributos completamente.
DA_TIPO_HABITACION_POR_HOTEL	Si	PK compuesta (ID_Hotel, ID_Tipo_Habitacion) determina Precio_Alta, Precio_Baja y Cantidad. Sin dependencias parciales.
DA_HABITACION	Si	ID_Habitacion determina todos los atributos. Sin dependencias transitivas.
DA_SERVICIO_COSTO_FIJO	Si	ID_Servicio determina todos los atributos. Sin anomalias.
DA_SERVICIO_COSTO_CONSUMO	Si	ID_Servicio determina todos los atributos. Sin anomalias.
DA_HOTEL_TIPO_HAB_SERVICIO	Si	PK compuesta es el unico determinante. Sin atributos no clave.

DA_CLIENTE	Si	Documento_Identidad determina todos los atributos. Sin dependencias parciales ni transitivas.
DA_ADMINISTRADOR	Si	Documento_Identidad determina todos los atributos. Sin dependencias parciales ni transitivas.
DA_ADMINISTRADO_POR	Si	PK compuesta es el unico determinante. Sin atributos no clave.
DA_RESERVAS	Si	ID_Reserva determina todos los atributos. FKs no generan dependencias transitivas.
DA_RESERVA_SERVICIO	Si	PK compuesta determina Cantidad, Noches y Personas_Servicio. Sin dependencias parciales.

Adicionalmente, en el repositorio entregado se encuentra nuestra primera entrega en caso de querer consultarla :D

Modelo E/R:



2. Implementación y esquema de base de datos:

Archivo de SQL script en el cual se evidencian la creación de las tablas en un orden especifico debido a la creación de llaves foráneas, script adjuntado en los documentos de entrega.

3. Población de Tablas:

Para poblar las 13 tablas que tenemos la siguiente base de datos de prueba completa y con las siguientes entidades, cada una está explicada con cada atributo y con su cantidad de datos totales:

DA_CIUDAD

Cantidad: 20

ID_CIUDAD: cada ciudad tiene su respectivo Id, este se puso con 6 cifras aleatorias ya que para que sea más apegado a la realidad se tuvo en cuenta que cada ciudad del mundo podría tener su propio Id, entonces se necesita una combinación larga de números asignados

NOMBRE: Los nombres de estas ciudades son los de las ciudades colombianas más principales y turísticas.

DA_ADMINISTRADOR

Cantidad: 50

DOCUMENTO_IDENTIDAD: Tal y como los documentos de identidad colombianos, cada uno es una serie al azar de 10 números-

NOMBRE: Es un solo nombre al azar de entre los nombres más comunes de Latinoamérica

APELLIDOS: Dos apellidos comunes en Latinoamérica de forma aleatoria.

CORREO_ELECTRONICO: Una dirección de correo electrónico institucional o personal ficticia, construida con el nombre, apellido y un número aleatorio, utilizando dominios como gmail.com, hotmail.com o outlook.com.

TELEFONO: Un número de contacto de 10 dígitos.

DA_CLIENTE

Cantidad: 300

DOCUMENTO_IDENTIDAD: Al igual que los administradores, consiste en una serie de 10 números aleatorios que identifican de forma única a cada cliente.

NOMBRE: Un nombre propio seleccionado al azar entre los más comunes de la región (ej. Carlos, Isabella, Juan, Valentina).

APELLIDOS: Dos apellidos comunes en Latinoamérica, generados de forma aleatoria (ej. Diaz Diaz, Gomez Lopez).

CORREO_ELECTRONICO: Una dirección de correo electrónico institucional o personal ficticia, construida con el nombre, apellido y un número aleatorio, utilizando dominios como gmail.com, hotmail.com o outlook.com.

TELEFONO: Un número de contacto de 10 dígitos.

DA_HOTEL

Cantidad: 20

ID_HOTEL: Un identificador numérico secuencial del 1 al 20.

NOMBRE: Nombres comerciales ficticios de hoteles.

DIRECCION: Ubicación física del hotel en su respectiva ciudad.

TELEFONO: Número de contacto telefónico fijo o celular.

NUMERO_HABITACIONES: Cantidad total de habitaciones que posee el hotel.

ID_CIUDAD: Llave foránea que vincula el hotel con una de las 20 ciudades registradas en `DA_CIUDAD`.

DA_TIPO_HABITACION

Cantidad: 50

ID_TIPO_HABITACION: Código identificador de 3 dígitos generado de forma aleatoria.

TIPO: Clasificación de la habitación (ej. Estándar, Suite, Doble, Lujo).

DIMENSIONES_HABITACION: Es el tamaño de la habitación medido en metros cuadrados

TIPO_CAMAS: Tipo de cama dependiendo de su tamaño

CANTIDAD_CAMAS: El número de camas disponibles en la habitación

TIPO_VISTA: El tipo de vista de la ventana de la habitación, Estándar, A piscina, A monumentos o al mar

COMODIDADES_HABITACION: Detalle breve sobre las comodidades o características del tipo de habitación.

PRECIO: El precio de la noche en la habitación.

DA_TIPO_HABITACION_POR_HOTEL

Cantidad: 101

ID_HOTEL / ID_TIPO_HABITACION: Llaves foráneas que relacionan qué tipos de habitaciones ofrece cada hotel.

PRECIO_ALTA / PRECIO_BAJA: Costos de la habitación según la temporada turística.

CANTIDAD_HABITACIONES_DISPONIBLES: Número de unidades de ese tipo específico que tiene el hotel.

DA_HABITACION

Cantidad: 1665

ID_HABITACION: Un identificador único de 6 cifras para cada habitación individual.

NUMERO_HABITACION: El número comercial o de puerta.

ID_HOTEL: Indica a qué hotel pertenece la habitación.

ID_TIPO_HABITACION: Clasifica la habitación según los tipos existentes.

DA_SERVICIO_COSTO_FIJO

Cantidad: 20

ID_SERVICIO: Identificador único del servicio enumerado del 1 al 20 según orden de creación.

NOMBRE: Servicios con costo estándar (ej. WiFi Premium, Parqueadero Privado, Seguro Hotelero).

APLICA: indica el tipo del servicio fijo, según por reserva, por persona o por noche

TIPO_COSTO: todos los de esta columna son fijos por defecto

TIPO_SERV_DISPONIBLE: Es el lugar en el cual se hará efectivo el cumplimiento del servicio

DESCRIPCION_SERVICIO: es una descripción de las ventajas que ofrece el servicio

PRECIO: Valor monetario establecido para el servicio.

DA_SERVICIO_COSTO_CONSUMO

Cantidad: 50

ID_SERVICIO: Identificador único del servicio enumerado del 1 al 50 según orden de creación.

NOMBRE: Servicios con costo estándar

APLICA: indica el tipo del servicio fijo, según por reserva, por persona o por noche

TIPO_COSTO: todos los de esta columna son de consumo por defecto

TIPO_SERV_DISPONIBLE: Es el lugar en el cual se hará efectivo el cumplimiento del servicio

DESCRIPCION_SERVICIO: es una descripción de las ventajas que ofrece el servicio

PRECIO_UNIDAD: Valor monetario establecido para cada servicio individualmente.

DA_HOTEL_TIPO_HAB_SERVICIO

Cantidad: 300

ID_HOTEL: Llave foránea que referencia al hotel específico.

ID_TIPO_HABITACION: Llave foránea que indica el tipo de habitación dentro de ese hotel.

ID_SERVICIO_FIJO: Referencia a un servicio de costo estándar (como seguro o WiFi) asociado a esa habitación.

ID_SERVICIO_CONSUMO: Referencia a un servicio por uso (como minibar o lavandería) disponible para esa configuración.

DA_ADMINISTRADO_POR

Cantidad: 59

ID_HOTEL: Identificador del hotel que está siendo gestionado.

DOCUMENTO_IDENTIDAD: Documento del administrador asignado. Esta tabla permite que un hotel tenga varios administradores (por turnos o sedes) y que un administrador pueda supervisar más de un hotel.

DA_RESERVAS

Cantidad: 10

ID_RESERVA: Número secuencial único para cada transacción de hospedaje.

FECHA_INICIO / FECHA_FINAL: Registra el periodo de estancia del cliente.

PRECIO_TOTAL_RESERVA: Monto calculado por el tiempo de estancia y el tipo de habitación.

ADULTOS / MENORES_3_ANIOS: Cantidad de personas desglosadas por edad para control de capacidad y tarifas.

ID_HOTEL / DOCUMENTO_IDENTIDAD / ID_TIPO_HABITACION: Llaves foráneas que conectan la reserva con el establecimiento, el cliente que paga y el tipo de alojamiento elegido.

DA_RESERVA_SERVICIO

Cantidad: 16

ID_RESERVA: Es la llave foránea que conecta este registro con la tabla DA_RESERVAS. Indica a qué reserva específica se le están cargando los servicios adicionales.

ID_SERVICIO_FIJO: Identificador del servicio de costo estándar (como WiFi o Seguro Hotelero) que el cliente decidió tomar para su estancia.

ID_SERVICIO_CONSUMO: Identificador del servicio que se cobra por uso (como minibar o lavandería) consumido durante la reserva.

CANTIDAD: Representa el número de veces que se utilizó el servicio de consumo (por ejemplo, 3 kits dentales o 2 servicios de lavandería).

NOCHES: Indica el número de noches durante las cuales se aplicaron los servicios, especialmente útil para aquellos servicios que se cobran por jornada.

PERSONAS_SERVICIO: Especifica para cuántas personas de la reserva se solicitó el servicio, teniendo en cuenta que una familia en Colombia tiene un promedio de 2.84 personas y que el 90% de los hoteles reciben reservas de familias

4. Diseño y Sentencias de SQL:

a) Para cada consulta, la documentación que describe las tablas usadas, los atributos de dichas tablas para realizar los joins necesarios de la consulta, y una corta explicación de por qué se escogió el tipo de join usado:

Req1

Se utilizan las tablas de tipos de habitación por hotel y reservas, relacionándolas por ID_HOTEL y ID_TIPO_HABITACION, además de una condición de fechas. Se emplea un LEFT JOIN para asegurar que se incluyan todos los tipos de habitación del hotel, incluso aquellos sin reservas en el rango consultado, lo que permite calcular correctamente la disponibilidad comparando contra el conteo de reservas.

Req2

Se combinan reservas, hoteles y ciudades mediante sus llaves foráneas (ID_HOTEL y ID_CIUDAD) para obtener el número de reservas por mes, hotel y ciudad. Se utiliza INNER JOIN porque solo interesan registros completos y válidos donde cada reserva esté asociada a un hotel y este a una ciudad.

Req3

Se relacionan reservas, hoteles y ciudades usando INNER JOIN para calcular el total de huéspedes (adultos y menores) por fecha, hotel y ciudad. Este tipo de join garantiza que solo se consideren reservas válidas con información completa, evitando registros inconsistentes.

Req4

Se unen reservas, hoteles y ciudades mediante INNER JOIN para calcular los ingresos totales por hotel y ciudad en un rango de fechas. Se usa este tipo de join porque el análisis requiere únicamente datos consistentes y completamente relacionados entre las tablas.

Req5

Se combinan reservas, hoteles y servicios, donde los servicios pueden ser de costo fijo o de consumo. Se usan INNER JOIN para las relaciones obligatorias y LEFT JOIN para las tablas de servicios, ya que cada registro puede pertenecer a solo uno de los dos tipos. Esto permite no perder información y unificar los nombres mediante COALESCE, luego en la siguiente se relacionan reservas, hoteles y tipos de habitación mediante INNER JOIN para contar cuántas veces se ha reservado cada tipo de habitación por hotel. Este join se utiliza porque solo interesan registros válidos y completamente definidos para el análisis.

5. Desarrollo de la aplicación:

Arquitectura de la aplicación:

En este caso, las páginas encapsulan la vista (regiones, items, botones), la lógica de negocio se implementa mediante procesos PL/SQL del lado del servidor, y el modelo de datos reside en Oracle dado que esta fue la plataforma indicada en las clases y en el enunciado.

Roles:

Se definen dos roles principales dentro de la aplicación:

- Administrador: accede a todos los requerimientos funcionales (RF1–RF8) y a todos los requerimientos funcionales de consulta (RFC1–RFC5). Las páginas de consulta solo son visibles cuando el usuario autenticado posee el rol admin.
  USUARIO ADMIN: ADMIN
  CONTRASEÑA: ADMIN
- Cliente: puede registrarse (RF5), gestionar su cuenta (RF6), realizar reservas (RF7) y gestionar sus reservas (RF8). Las páginas RFC son invisibles para este rol.

USUARIO CLIENTE: CLIENTE
CONTRASEÑA: CLIENTE

El control de acceso se implementó a través de:

- **Authorization Schemes** configurados por página para restringir el acceso según rol.
- Server-Side Conditions en regiones e ítems para mostrar u ocultar elementos según el usuario autenticado (lo cual fue lo que más se nos dificultó).
- Redirecciones automáticas hacia la página de login cuando un usuario no autenticado intenta acceder a páginas protegidas.

Estructura de las páginas creadas:

RFC	Nombre	Pág.	Items de página
RFC1	Habitaciones Disponibles	Pág. 2	P2_FECHA_INICIO, P2_FECHA_FIN, P2_ID_HOTEL
RFC2	Reservas en el Tiempo	Pág. 3	P3_FECHA_INICIO, P3_FECHA_FIN, P3_ID_CIUDAD, P3_ID_HOTEL
RFC3	Huéspedes por Fecha	Pág. 4	P4_FECHA_INICIO, P4_FECHA_FIN, P4_ID_CIUDAD, P4_ID_HOTEL
RFC4	Ingresos por Hotel	Pág. 5	P5_FECHA_INICIO, P5_FECHA_FIN
RFC5	Servicios y Tipos Populares	Pág. 6	P6_FECHA_INICIO, P6_FECHA_FIN

RF	Nombre	Rol	Descripción
RF1	Registrar Ciudad	Administrador	Permite dar de alta una nueva ciudad con nombre único
RF2	Registrar Hotel	Administrador	Asocia hotel a una ciudad existente; campos: nombre, dirección, teléfono, descripción
RF2.5	Registrar Servicio	Administrador	Define servicios con tipo de costo (fijo / por consumo)

RF3	Registrar Habitaciones	Administrador	Incluye tipo, camas, costos, cantidad disponible y servicios adicionales por hotel
RF4	Editar Habitaciones	Administrador	Edición in-line de nombre, costo, camas y amenidades; aplica por hotel de forma independiente
RF5	Creación de cuenta	Cliente / Admin	Accesible sin autenticación; guarda ID, nombre, correo, teléfono
RF6	Gestión de cuenta	Cliente	El cliente modifica sus propios datos; validado por sesión
RF7	Realizar reserva	Cliente	Verifica disponibilidad, calcula precio (temporada alta/baja), adjunta servicios adicionales
RF8	Gestionar reserva	Cliente	Consulta historial; modifica fechas o cancela si faltan ≥ 3 días para el check-in

Justificación requerimientos funcionales de consulta (RFC):

Cada RFC se implementó en APEX como una región de tipo Classic Report o Chart cuyo origen es una SQL Query parametrizada. Los filtros se exponen mediante Date Pickers y Select Lists vinculados a los bind variables del query. El botón Buscar tiene configurado el campo 'Page Items to Submit' con todos los ítems de filtro de su página, de modo que al hacer clic se refresca la región con los valores actualizados.

Todas las Querys utilizadas acá se encuentran en sus scrpits respectivos. Estas se modificaron un poco a las originales hechas en el punto anterior dado que en algunos casos los Date Picker molestaron, o las Querys de APEX no nos dejaban ejecutar las originales, o sucedían problemas de respuesta con las originales, etc. Por lo que se modificaron un poco para su funcionalidad.

RFC1 – Habitaciones Disponibles

Muestra las habitaciones disponibles en un hotel para un intervalo de fechas. El usuario selecciona el hotel y las fechas de entrada/salida. La consulta devuelve el tipo de habitación, el total disponible, las reservas solapadas con el período y las habitaciones libres resultantes.

RFC2 – Reservas en el Tiempo

Permite visualizar la evolución mensual de reservas en un rango de fechas, filtrable por ciudad y hotel. Agrupa los resultados por mes (formato MM/YYYY), hotel y ciudad, facilitando la identificación de temporadas con mayor flujo.

RFC3 – Huéspedes por Fecha y Ciudad

Muestra el total de huéspedes (adultos + menores de 3 años) por fecha, ciudad y hotel. Permite filtrar por ciudad y por hotel específico para analizar la afluencia diaria en cada establecimiento.

RFC4 – Ingresos por Hotel

Calcula los ingresos totales generados por cada hotel en un rango de fechas, ordenados de mayor a menor. Permite comparar el rendimiento económico entre hoteles y ciudades.

RFC5 – Servicios y Tipos de Habitación Populares

Identifica los tipos de habitación más reservados por hotel en un período dado. La región 'Tipos de Habitación más reservados' usa COUNT(*) agrupado por hotel y tipo; una segunda región (Servicios más usados) usa la misma lógica sobre la tabla de servicios asociados a reservas.

Justificación de los requerimientos funcionales (RF) - Lógica de implementación:

Los requerimientos funcionales se implementaron principalmente mediante los mecanismos nativos de APEX, sin necesidad de escribir código PL/SQL extenso, por lo que explicaremos la lógica utilizada y algunos errores que tuvimos.

RF1 – Registrar Ciudad

Página con Form Region conectada a DA_CIUDAD. APEX genera automáticamente los procesos de INSERT/UPDATE. Se añadió una restricción UNIQUE sobre NOMBRE_CIUDAD en la base de datos, de modo que al intentar insertar una ciudad duplicada APEX muestra el error ORA-00001.

RF2 – Registrar Hotel

Form Region conectada a DA_HOTEL con un Select List para escoger la ciudad (población dinámica desde DA_CIUDAD). La FK ID_CIUDAD garantiza integridad referencial a nivel de base de datos.

RF2.5 – Registrar Servicio

Form Region conectada a DA_SERVICIO. Incluye un Radio Group para el tipo de costo (FIJO / POR_CONSUMO) y un campo numérico para el precio. Una restricción CHECK en la base de datos valida el tipo de costo.

RF3 – Registrar Habitaciones para un Hotel

Formulario multi-sección que crea un registro en DA_TIPO_HABITACION_POR_HOTEL asociando un tipo de habitación a un hotel. Incluye Checkbox Group para las amenidades y servicios adicionales disponibles. La disponibilidad se registra como cantidad numérica.

RF4 – Editar Habitaciones

Interactive Grid sobre DA_TIPO_HABITACION_POR_HOTEL filtrada por el hotel seleccionado. Permite edición in-line de nombre, costo temporada alta/baja, tipo y cantidad de camas, y amenidades. Los cambios se aplican únicamente al registro del hotel activo, sin afectar otros hoteles con el mismo tipo de habitación.

RF5 – Creación de Cuenta de Cliente

Página pública (sin Authorization Scheme) con Form Region sobre DA_CLIENTE. Accesible tanto para usuarios no autenticados como para el administrador. El correo electrónico tiene restricción UNIQUE en la base de datos.

RF6 – Gestión de Cuenta de Cliente

Form Region que precarga los datos del cliente autenticado usando :APP_USER como llave de búsqueda. El cliente solo puede ver y modificar su propio registro; un Server-Side Condition impide el acceso a registros de otros clientes.

RF7 – Realizar Reserva

Implementado como un formulario de múltiples pasos: selección de hotel y fechas, selección de tipo de habitación con verificación de disponibilidad, selección de servicios adicionales, cálculo del precio total considerando temporada alta/baja y número de noches, confirmación y generación de código de reserva. La disponibilidad se verifica

comparando las reservas solapadas contra la cantidad de habitaciones disponibles del tipo seleccionado.

RF8 – Gestionar Reserva

Classic Report con el historial de reservas del cliente autenticado. La condición de negocio más relevante se implementó con un Server-Side Condition en PL/SQL:

Condición para habilitar Cancelar / Modificar RETURN (TRUNC(:FECHA_INICIO) - TRUNC(SYSDATE)) >= 3;

Esta expresión garantiza que el cliente solo pueda cancelar o modificar la reserva si faltan al menos 3 días para el inicio de la estadía.

6. Escenarios de Prueba:

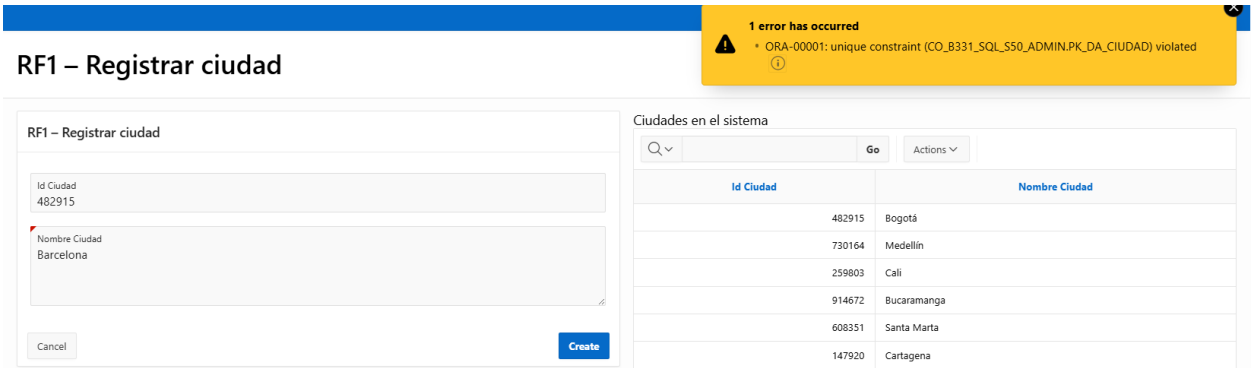
Caso 1: Unicidad.

Objetivo: Validar unicidad de tuplas por PK.
Precondiciones: Tabla Ciudad vacía o sin Id_Ciudad = 10.
Entradas:

- Ciudad1: Id_Ciudad = 482915, Nombre_Ciudad = "Bogotá"
- Ciudad2: Id_Ciudad = 482915, Nombre_Ciudad = "Barcelona"

Pasos:

1. Insertar Ciudad1.
2. Insertar Ciudad2.



Resultado :

- Paso 1: Se inserta.
- Paso 2 : Falla por PK duplicada (Id_Ciudad repetido).

Por lo que concluimos que la unicidad en la entidad Ciudad se cumple evitando tener duplicados de ciudades.

Caso 2 : Unicidad compuesta

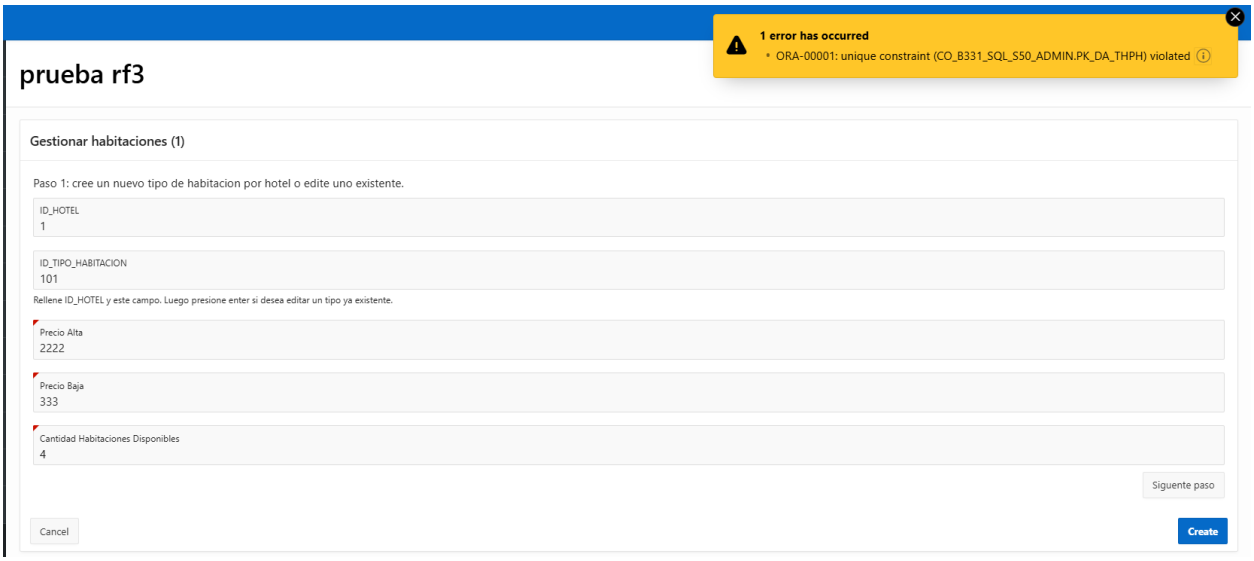
Objetivo: Validar PK compuesta (Por medio de Id_Hotel, Id_Tipo_Habitación).
Precondiciones: Existe Hotel (Id_Hotel = 1) y Tipo_Habitación(

Id_Tipo_Habitación = 2).
Entradas:

- Registro1: (1, 2, Precio_Alta = 300, Precio_Baja = 200, Cantidad = 5)
- Registro2: (1, 2, Precio_Alta = 350, Precio_Baja = 250, Cantidad = 6)

Pasos:

1. Insertar Registro1.
2. Insertar Registro2 con misma PK compuesta (1, 2)



Resultado :

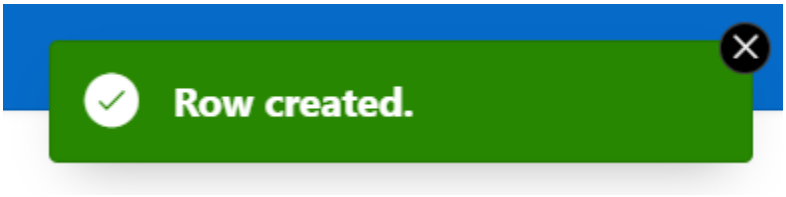
- Paso 1: Se inserta.
- Paso 2: Falla por PK compuesta duplicada.

Por lo que podemos concluir que por medio de la entidad intermedia ayudamos a evitar duplicados en un mismo tipo de habitacion pero con diferentes cantidades y precios. Esto evita confusiones del Administrador, adicionalmente, posibles confusiones del cliente.

Caso 3 : FK válida

Objetivo: Probar inserción con FK existente.
Precondiciones: Existe Ciudad(Id_Ciudad= 482915).

Entradas: Hotel(Id_Hotel=100, Id_Ciudad= 482915, ...)



Pasos: Insertar el hotel.
Resultado : Inserta correctamente.

Este tipo de “conexión” la logramos por medio de las FK validas en las entidades y relaciones del modelo con una llave ya existente.

Caso 4 : FK inválida

Objetivo: Probar inserción con FK inexistente.
Precondiciones: No existe Ciudad(Id_Ciudad=999).

Entradas: Hotel(Id_Hotel=101, Id_Ciudad=999, ...)

RF2 – Registrar Hotel

Pasos: Insertar el hotel.
Resultado esperado: Falla por violación FK. Directamente no se puede agregar nuestra app solo permite ciudades registradas

Al no existir la llave dado que no existe el id 999 no logramos registrar el hotel a una debida ciudad. Esto hace que el Administrador evite tener "hoteles fantasma" los cuales no tienen un lugar físico (Sí, esto ayuda a ser un poco más legales jaja).

Caso 5 : CHECK de fechas

Objetivo: Validar que > Fecha_inicio.
Precondiciones: Existe Cliente, Hotel y Tipo_Habitación referenciables.
Entradas:

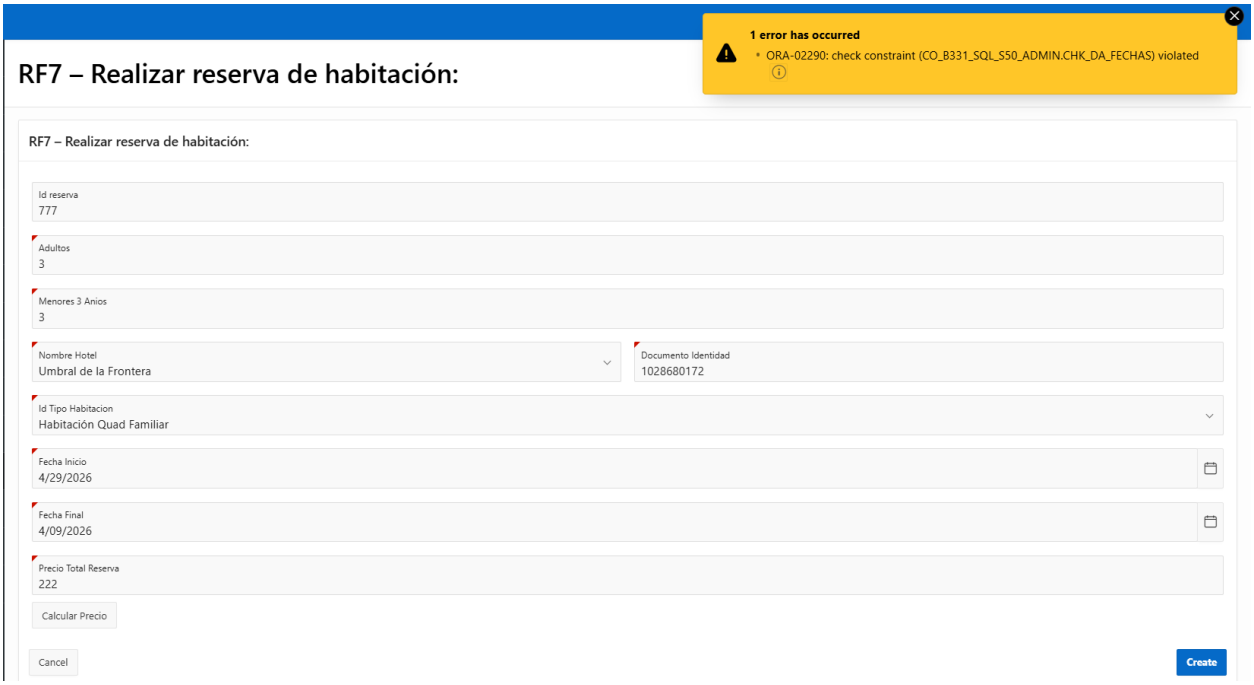
- Reserva válida: inicio=2026-03-10, final=2026-03-12
- Reserva inválida: inicio=2026-03-10, final=2026-03-10

Pasos:

1. Insertar reserva válida.
2. Insertar reserva inválida.

Resultado :

- Paso 1: Se insertan las fechas.
- Paso 2 : Falla por CHECK.



Esto significa que queremos evitar reservas tengan las mismas fechas. Es decir, para un cliente reserva mínimo una noche.

Caso 6: Validación de precios

Objetivo: No permitir precios negativos.
Precondiciones: Existe Hotel=1 y Tipo=2.
Entradas:

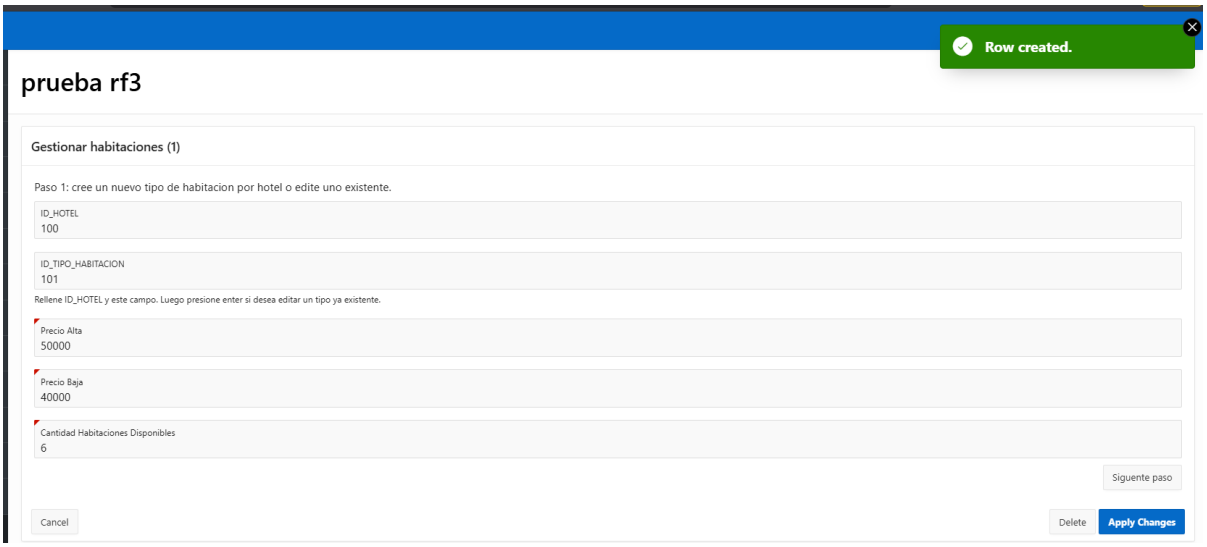
- Configuracion válida: Precio_Alta=300, Precio_Baja=200
- Configuración inválida: Precio_Alta=-1

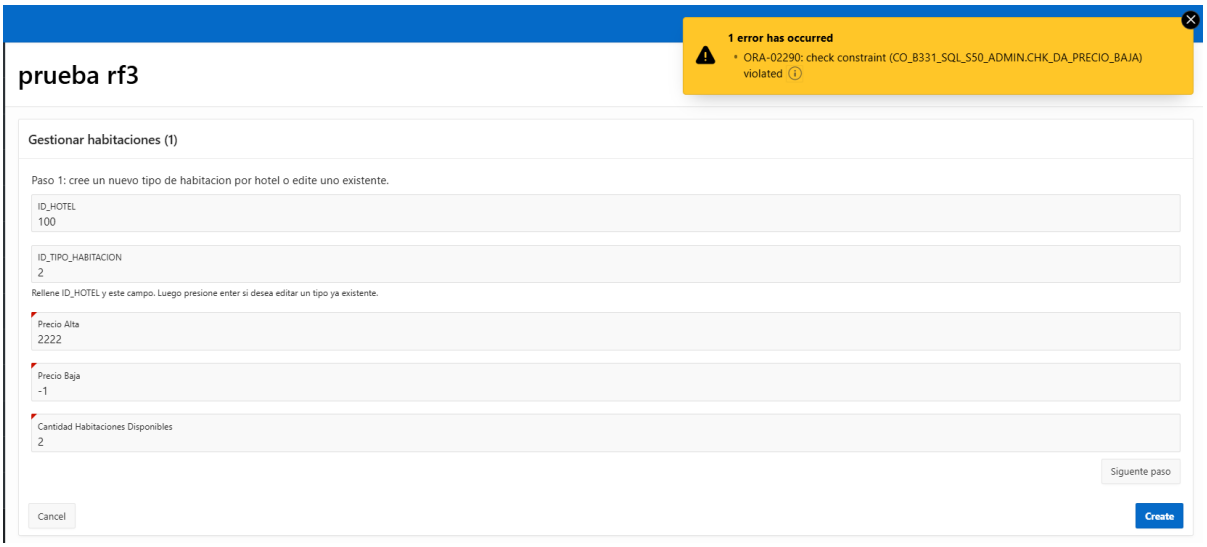
Pasos:

1. Insertar configuración válida en Tipo_Habitación_Por_Hotel.
2. Insertar configuracion inválida.

Resultado :

- Paso 1: Se inserta.

- 

- Paso 2: Falla por <=1
- 

En este caso, queremos que no se internen números negativos a los precios, pues de esta forma también se puede manipular la base datos. En caso de que la cadena hotelera maneje descuentos, se puede agregar a la base de datos, sin embargo, tener precios negativos solo agrega problemas a la hora de realizar cálculos en las ganancias de la empresa y de los precios presentados hacia el cliente.

- Los casos presentados a continuacion son desde la perspectiva de los diferentes actores del proyecto, es decir, de consulta y almacenamiento, por lo que pueden ser un poco redundantes con los presentados anteriormente.

Caso 8 : Registrar ciudad

Rol: Administrador
Precondiciones: No existe Id_Ciudad=20.

Entradas: Ciudad(20, “Cartagena”)

Pasos: Insertar ciudad.
Resultado esperado: Ciudad registrada y consultable.

Caso explicado anteriormente. Esta vez, desde la vista del administrador.

Caso 9 : Registrar hotel

Rol: Administrador
Precondiciones: Existe Ciudad(Id_Ciudad=20).

Entradas: Hotel(Id_Hotel=200, Nombre, Dirección, Teléfono, Id_Ciudad=20)

Pasos: Insertar hotel.
Resultado esperado: Hotel registrado en esa ciudad.

Caso explicado anteriormente. Esta vez, desde la vista del administrador.

Caso 10 : Configurar tipo por hotel

Rol: Administrador
Precondiciones: Existe Hotel=200 y Tipo_Habitación=2.

Entradas: Tipo_Habitación_Por_Hotel(Hotel=200, Tipo=2, Precio_Alta=400, Precio_Baja=300, Cantidad=8)

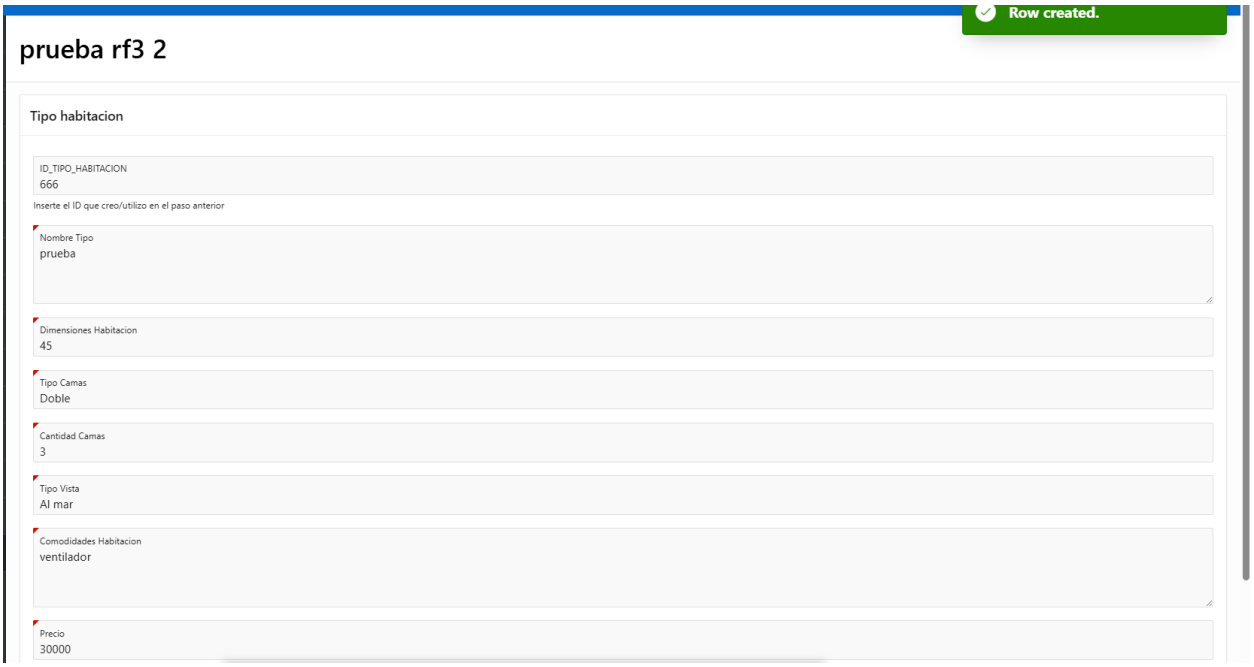


Table | Data | Indexes | Model | Constraints | Grants | Statistics | UI Defaults | Triggers | Dependencies | SQL | REST | Sample Queries

Query | Count Rows | Insert Row | Load Data

EDIT	ID_TIPO_HABITACION	NOMBRE_TIPO	DIMENSIONES_HABITACION	TIPO_CAMAS	CANTIDAD_CAMAS	TIPO_VISTA	COMODIDADES_HABITACION	PRECIO
	326	Habitación King Deluxe	45	King	1	Estandar	Cama extra grande, Aire acondicionado, Wifi, Minibar	490000
	327	Suite Penthouse	140	King	1	Al mar	Piso superior, Terraza privada, Jacuzzi, Aire acondicionado	3200000
	328	Doble Confort Plus	30	Doble	1	A piscina	Aire acondicionado, Wifi, Minibar, Televisión, Cafetera	415000
	329	Single Vista Histórica	21	Single	1	A monumentos	Ventanal antiguo, Calefacción, Wifi, Baño privado	360000
	330	Suite Familiar Marina	82	Doble	2	Al mar	Dos habitaciones, Aire acondicionado, Wifi, Cocina equipada	1450000
	666	prueba	45	Doble	3	Al mar	ventilador	30000

Download

Pasos: Insertar configuración.
Resultado esperado: El hotel queda ofreciendo ese tipo con precios y cupo.

Como podemos ver, el administrador puede insertar un tipo de habitacion en específico con una cantidad en especifica. Esto nos ayuda a tener más control sobre los datos y filtrado.

Caso 11: Asignar habitación física

Rol: Administrador
Precondiciones: Existe Hotel=200 y Tipo=2.

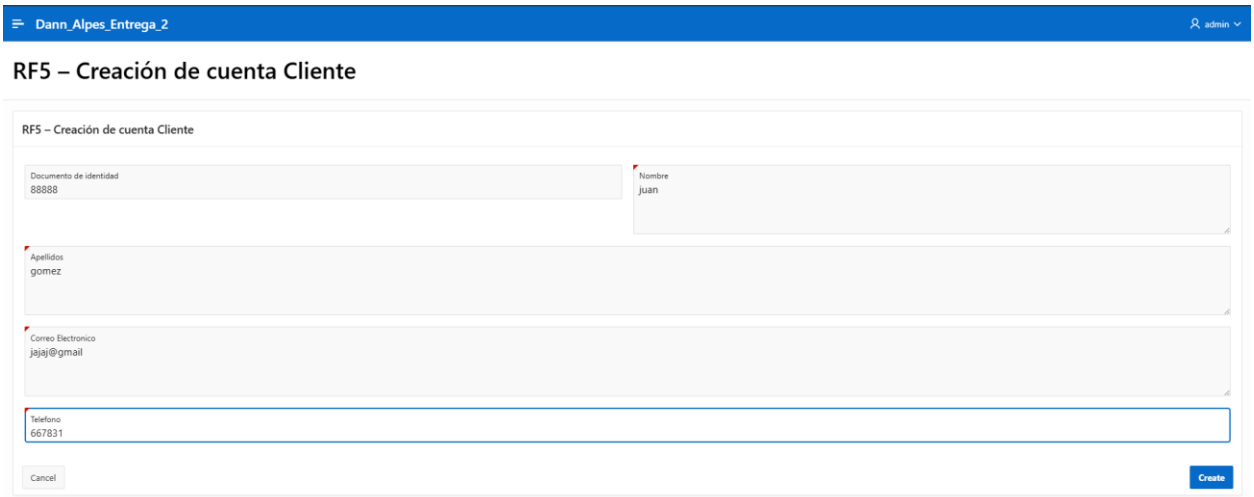
Entradas: Habitación (Id=900, Numero="203", Id_Hotel=200, Id_Tipo_Habitación=2)

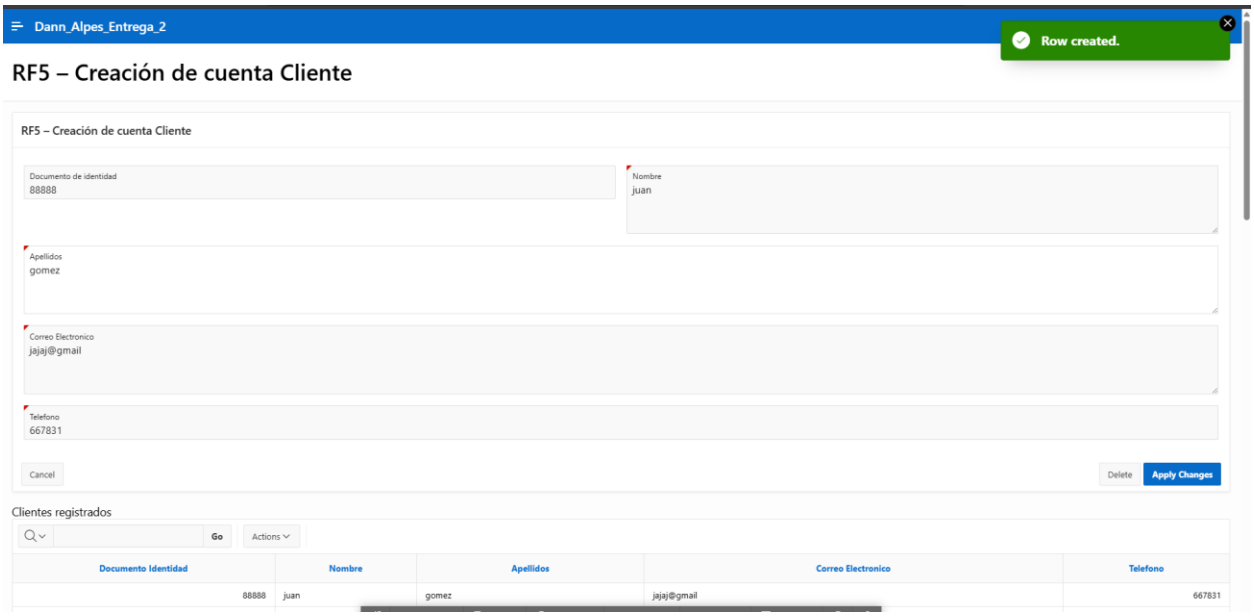
Pasos: Insertar habitación.
Resultado esperado: Habitación queda asociada al hotel y tipo.

Esto nos permite asignar un numero de habitacion en un hotel en específico, junto con su tipo asociado.

Caso 13: Crear cliente

Rol: Cliente o Administrador
Precondiciones: No existe Documento_Identidad="CC123".
Entradas: Cliente("CC123", Nombre, Apellidos, Correo, Teléfono)
Pasos: Insertar cliente.
Resultado esperado: Cliente registrado.

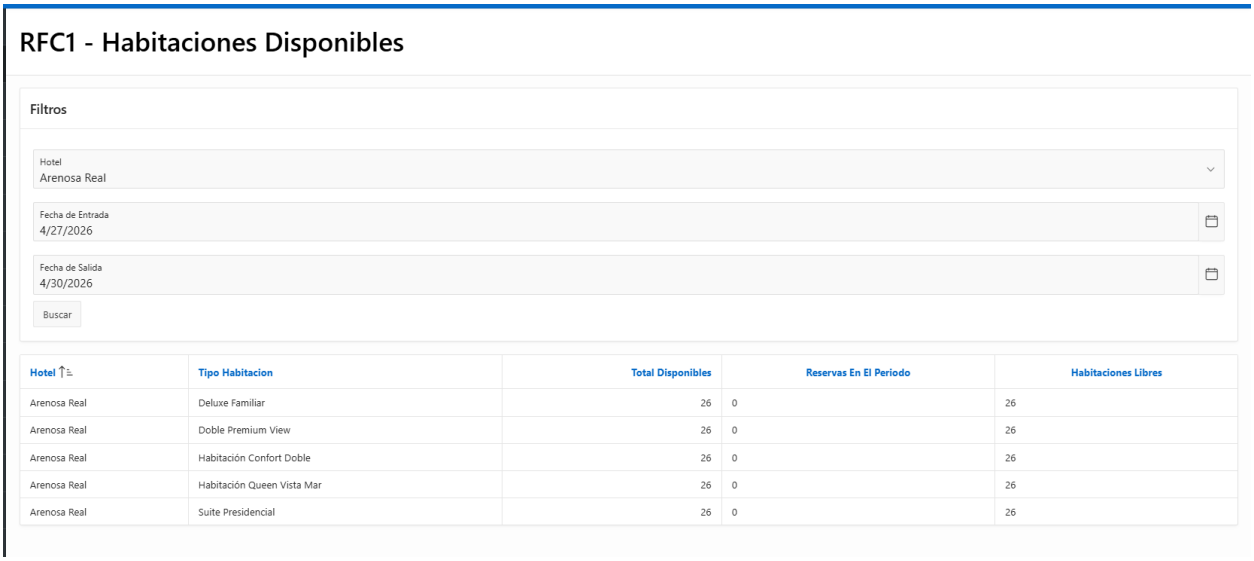




Gracias a que la base de datos le pide al cliente toda la información necesaria para hacer una reserva, el cliente se crea con éxito en la base de datos y está listo para reservar.

Caso 16: Consultar disponibilidad

Rol: Cualquier usuario (sin cuenta también)
Precondiciones: Existe Hotel=200. Existe cupo Cantidad_Habitaciones_Disponibles=2 para Tipo=2. Existe 1 reserva activa en ese rango para Tipo=2.



Hotel	Tipo Habitacion	Total Disponibles	Reservas En El Periodo	Habitaciones Libres
Arenosa Real	Deluxe Familiar	26	0	26
Arenosa Real	Doble Premium View	26	0	26
Arenosa Real	Habitación Confort Doble	26	0	26
Arenosa Real	Habitación Queen Vista Mar	26	0	26
Arenosa Real	Suite Presidencial	26	0	26

Pasos: Ejecutar consulta de disponibilidad.
Resultado: se puede ver que hay habitaciones disponibles.

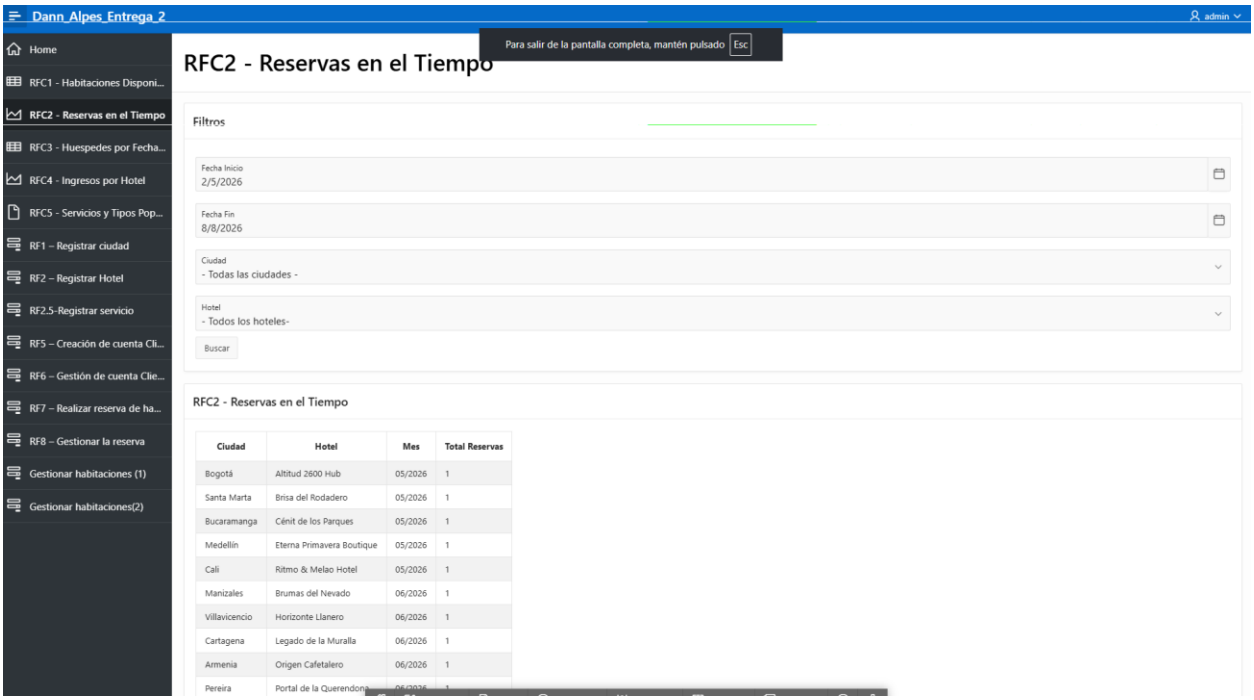
Por medio de las relaciones establecidas en el modelo, y FKs establecidas logramos este tipo de consultas especificas por fechas y tipos de habitacion, con la finalidad de saber si se encuentras disponibles o agotadas. Sin la ayuda de algunas entidades/tablas auxiliares la información estaría más regada de forma desorganizada y hacer etas consultas pueden llegar a ser un poco confusas.

Caso 17: Evolución de reservas por tiempo

Rol: Administrador
Precondiciones: Hay reservas en varios meses y hoteles.

Entradas: rango anual + filtro por ciudad/hotel

Pasos: Consultar conteos por periodo (mes).



Resultado : Serie temporal coherente con datos cargados.

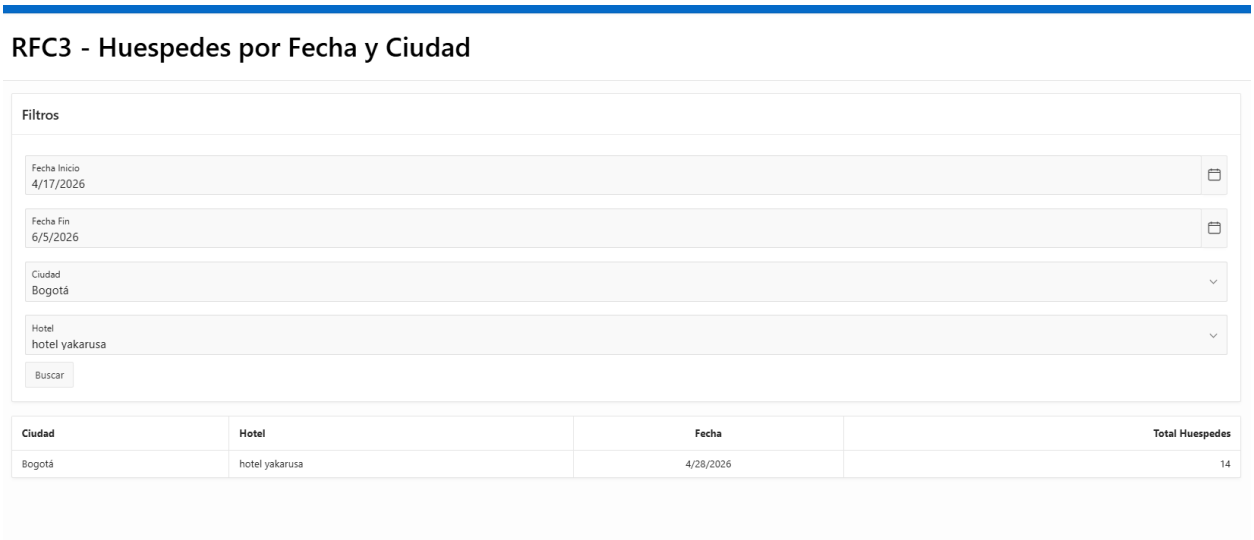
Todo esto lo podemos lograr por medio de filtrado en el modelo lógico y sus herramientas relacionadas. Por lo que la información es de fácil acceso para el Administrador, lo que permite hacer mejores análisis de, negocio.

Caso 18: Cantidad de huéspedes por fechas y ciudad

Rol: Administrador
Precondiciones: Reservas con Adultos y Menores

Entradas: fecha X, ciudad Y

Nuestra reserva:



Pasos: Sumar Adultos+Menores en reservas coincidentes.
Resultado : Total correcto por ciudad y por hotel.

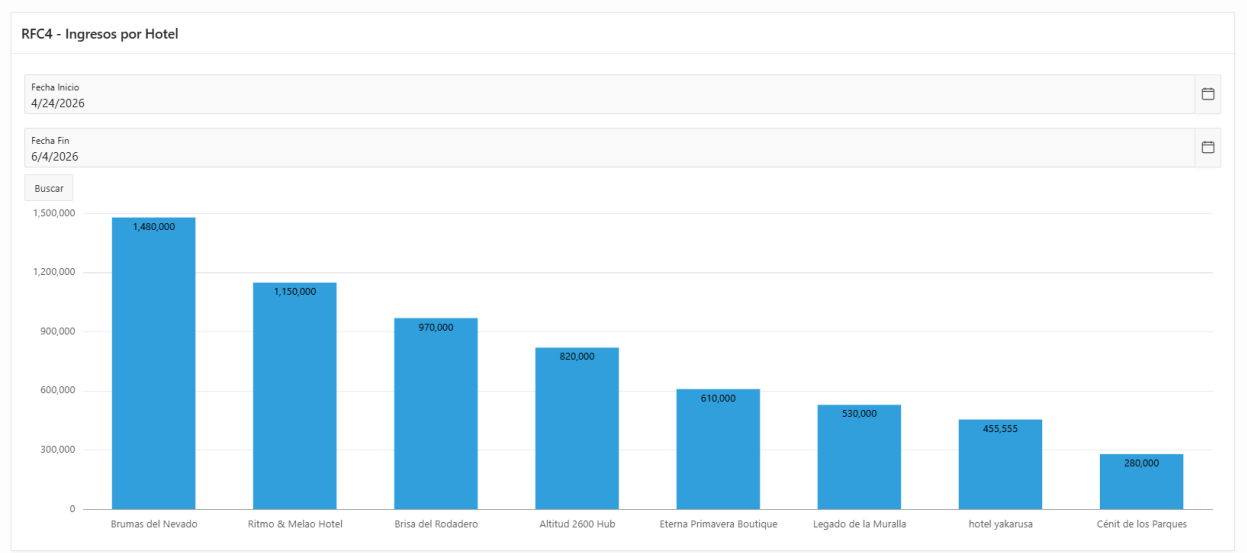
Al igual que el caso anterior, tenemos la información a la mano por medio del filtrado deseado. Solo queda realizar las operaciones deseadas por el Administrador por medio de las herramientas de los modelos.

Caso 19: Ingreso total por hotel

Rol: Administrador
Precondiciones: Reservas con Precio_Total_Reserva en rango.

Entradas: rango fechas

RFC4 - Ingresos por Hotel



Comprobando con las reservas:

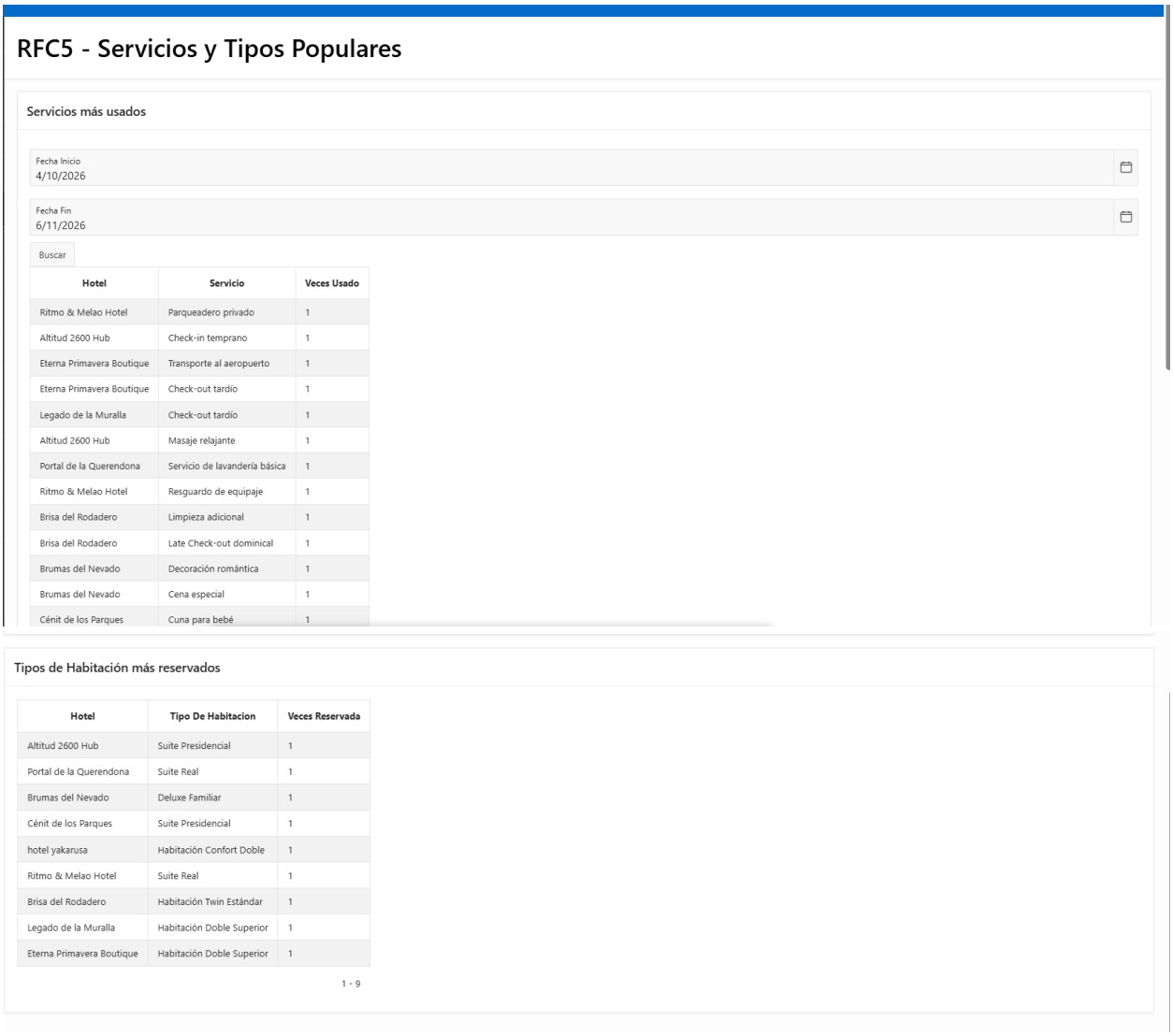
EDIT	ID_RESERVA	FECHA_INICIO	FECHA_FINAL	PRECIO_TOTAL_RESERVA	ADULTOS	MENORES_3_ANIOS	ID_HOTEL	DOCUMENTO_IDENTIDAD	ID_TIPO_HABITACION
	1	01-May-2026	05-May-2026	820000	2	1	1	2843140779	101
	2	03-May-2026	06-May-2026	610000	2	0	2	3050572345	102
	3	10-May-2026	15-May-2026	1150000	2	2	3	8309283382	201
	4	12-May-2026	14-May-2026	280000	1	0	4	4353758815	101
	5	15-May-2026	20-May-2026	970000	2	1	5	6224269167	202
	6	01-Jun-2026	04-Jun-2026	530000	2	0	6	8007079781	102
	7	03-Jun-2026	10-Jun-2026	1480000	2	2	7	5723821072	203
	8	08-Jun-2026	12-Jun-2026	700000	2	1	8	3107497791	201
	9	15-Jun-2026	18-Jun-2026	400000	1	0	9	4488542398	101
	10	20-Jun-2026	25-Jun-2026	1080000	2	2	10	2867699528	202
	3000	28-Apr-2026	08-May-2026	455555	7	7	100	88888	307

Pasos: Sumar: Precio_Total_Reserva, con condicion de hotel y ciudad.
Resultado : Totales correctos

Tal como uno de los requerimientos de consulta, por medio de un rango de fecha podemos acceder a las reservas efectuadas en ese año, luego, por medio de FK encontrar sus respectivos hoteles y ciudades, sumando así el resultado deseado del Administrador, el cual es el precio total de los ingresos (interpretados como reservas) en ese año.

Caso 23:

Prueba para RFC5



Hotel	Servicio	Veces Usado
Ritmo & Melao Hotel	Parqueadero privado	1
Altitud 2600 Hub	Check-in temprano	1
Eterna Primavera Boutique	Transporte al aeropuerto	1
Eterna Primavera Boutique	Check-out tardío	1
Legado de la Muralla	Check-out tardío	1
Altitud 2600 Hub	Masaje relajante	1
Portal de la Querendona	Servicio de lavandería básica	1
Ritmo & Melao Hotel	Resguardo de equipaje	1
Brisa del Rodadero	Limpieza adicional	1
Brisa del Rodadero	Late Check-out dominical	1
Brumas del Nevado	Decoración romántica	1
Brumas del Nevado	Cena especial	1
Cénit de los Parques	Cuna para bebé	1

Hotel	Tipo De Habitacion	Veces Reservada
Altitud 2600 Hub	Suite Presidencial	1
Portal de la Querendona	Suite Real	1
Brumas del Nevado	Deluxe Familiar	1
Cénit de los Parques	Suite Presidencial	1
hotel yakarusa	Habitación Confort Doble	1
Ritmo & Melao Hotel	Suite Real	1
Brisa del Rodadero	Habitación Twin Estándar	1
Legado de la Muralla	Habitación Doble Superior	1
Eterna Primavera Boutique	Habitación Doble Superior	1

Resultado: Devolvió valores correctos según lo que tenemos en nuestra BD.

NOTA: Evitamos algunos casos de prueba sin embargo estos pueden ejecutarse en la app y ver su comportamiento correcto.

7. Principal reto que tuvimos:

El reto más significativo fue la gestión de roles. APEX no ofrece un sistema de roles multi-tenant nativo, por lo que se resolvió mediante:

- Authorization Schemes basados en el valor almacenado en la sesión (APP_USER y una tabla de roles DA_USUARIO_ROL).
- Redirecciones automáticas en el proceso Post-Authentication para asignar el rol a la sesión al momento del login.